Assignment 2: Writing Networked Applications

Sabhya Arora (2023CS11177)
Rachit Bhalani (2023CS10961)
COL334/672: Computer Networks

September 8, 2025

1 Word Counting Client

1.1 Methodology

Client: The client requests k words at a time increasing offset from 0 one by one until EOF is reached. Word frequencies are computed locally.
Server: The server loads the words file in memory and returns the requested words as the requests are received
Experiments: We varied k across multiple runs and measured completion time.

1.2 Results

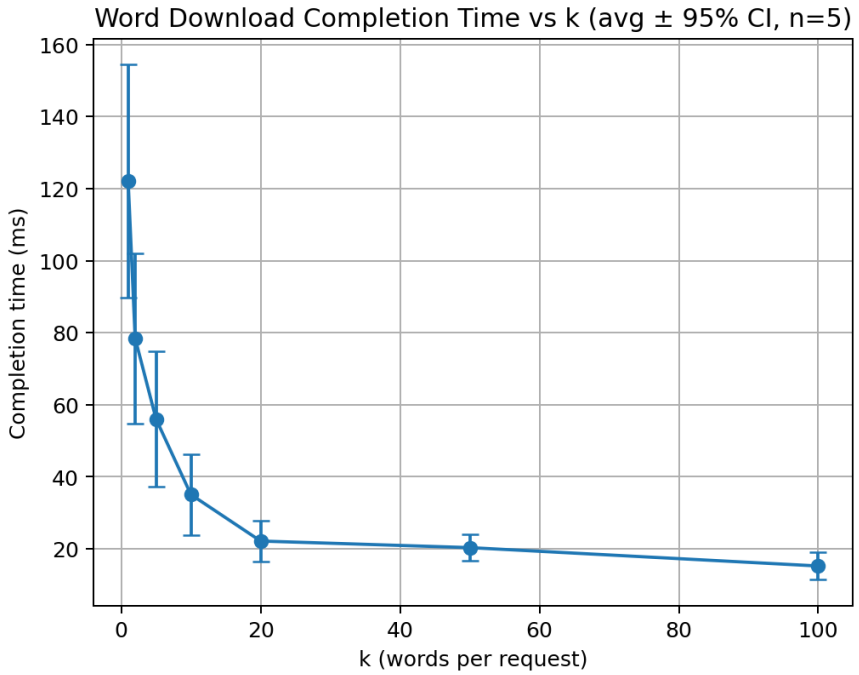


Figure 1: Completion time vs. k for word counting client. Error bars show 95% confidence intervals.

1.3 Discussion

When k is small, the client issues many requests, leading to high completion times due to per-request network overhead. As k increases, fewer requests are needed and completion time drops. Beyond a point, further increase in k does not yield benefits.

2 Concurrent Word Counting Clients

2.1 Methodology

Client: Now there are n different clients, each making their own file download requests to the server.
Server: We extended the server to support multiple concurrent clients via selectors. The requests returned by the selector API were handled in the order the sockets are returned by the API, hence implementing the first-come first-served scheduling policy at the server.
Experiments: Experiments were run with 1–32 clients in steps of 4, results averaged over 5 runs.

2.2 Results

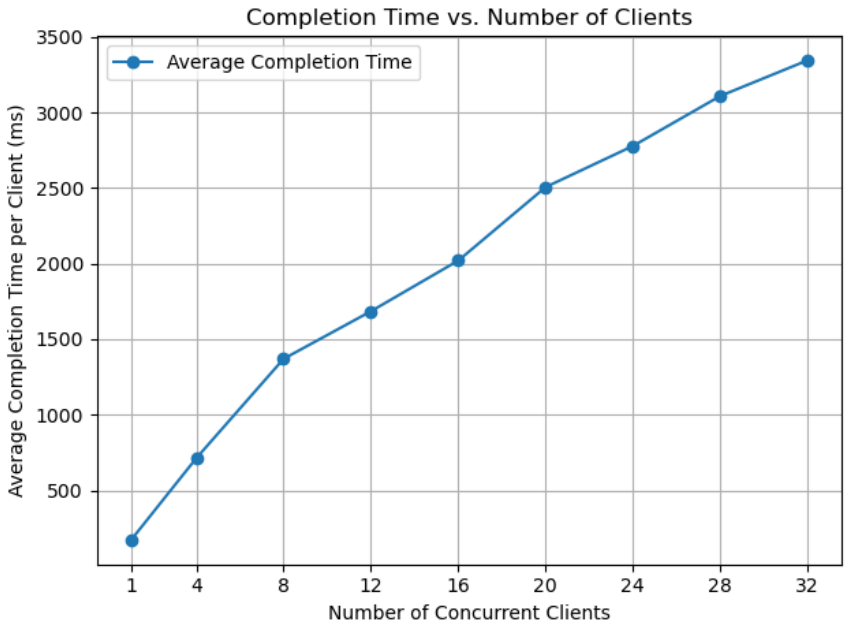


Figure 2: Average completion time per client vs. number of clients.

2.3 Discussion

As the number of clients increases, per-client completion time increases due to server multiplexing. The server processes only one client at a time and hence, other clients have to wait for the server to finish its processing.

3 When a Client Gets Greedy (FCFS Scheduling)

3.1 Methodology

Client: Now one of the n clients sends c requests at a time, that is, sends c requests and then waits for the response of these c requests from the server side.
Server: The server still processes requests in first-come first-served order.
Experiments: Experiments were run with 10 clients, with c ranging from 1 to 10, results averaged over 5 runs.

3.2 Results

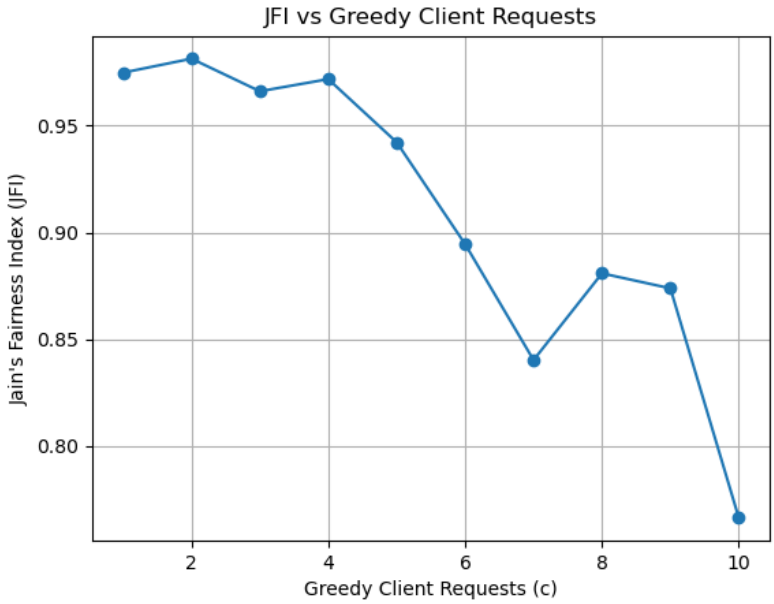


Figure 3: JFI vs. c for FCFS scheduling.

3.3 Discussion

Fairness degrades as c increases. With higher c, the greedy client monopolizes the server, leading to very low JFI. File download at the greedy client completes first. Other clients wait for the server to send a response, while the greedy client sends c responses at a time and then waits for response. At the server side, these requests may be processed simultaneously, and in such a case, the client would receive the response of all c requests simultaneously.
As c increases further, other clients will get an even lesser chance for the server to read their requests and file download will complete at the greedy client when only a few requests of other clients will be served. If we take the per-client "share" measure as

$$x_i = \frac{1}{\text{completion time}_i},$$

then Jain's Fairness Index (JFI) for N clients is

$$\text{JFI} = \frac{\left(\sum_{i=1}^{N} x_i\right)^2}{N \sum_{i=1}^{N} x_i^2}.$$

Assume the greedy client's completion time is t, while the remaining $N-1$ (normal) clients each take approximately $10t$ (Assuming the server processes 1 file download in time t). Then

$$x_{\text{greedy}} = \frac{1}{t}, \qquad x_{\text{normal}} = \frac{1}{10t}.$$

For the common case $N = 10$ (one greedy + nine normal clients) this evaluates to

$$\text{JFI} = \frac{\left(1 + \frac{9}{10}\right)^2}{10\left(1 + \frac{9}{100}\right)} = \frac{1.9^2}{10 \times 1.09} = \frac{3.61}{10.9} \approx 0.331.$$

So in this simplistic model, with the greedy client finishing in time t and others roughly $10t$, JFI falls to about 0.33.

4 When the Server Enforces Fairness (Round Robin)

4.1 Methodology

Client: The clients stays the same
Server: The server first assigns clients an id based on the order in which connections are established with them,i.e., TCP handshake occurs. Now, the server strictly processes requests in order of the client ids thus implementing a non-work conserving RR by waiting for the next client to be served to send its request and only then moving to the next client.
Experiments: Experiments were run with 10 clients, with c ranging from 1 to 10, results averaged over 5 runs.

4.2 Results

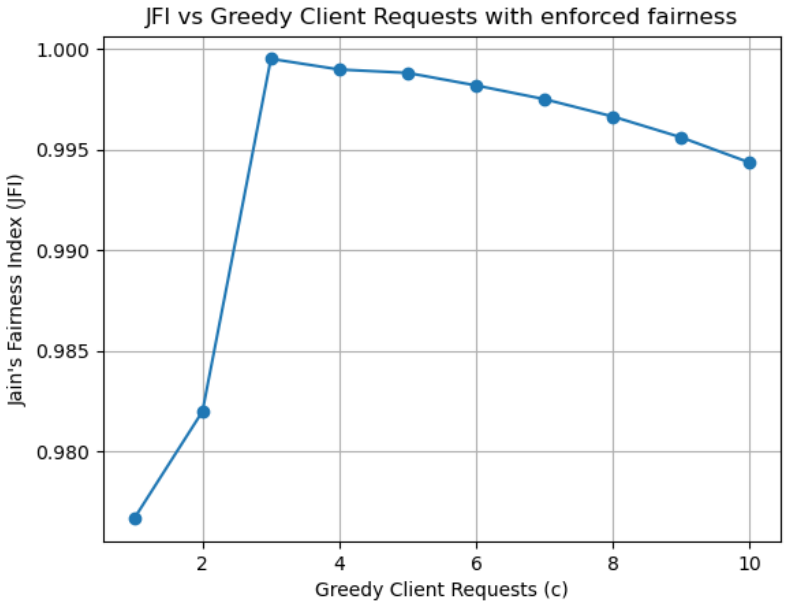


Figure 4: JFI vs. c for round robin scheduling.

4.3 Discussion

With round robin scheduling enforced, JFI stays close to 1 even at large c values. This is because now file download is completed for all clients at almost the same time because of non-work conserving RR implementation at the server.
A rogue client may still monopolize the server by

- Opening multiple connections to the server - one connection would be treated as one RR slot
- By making its requests heavy